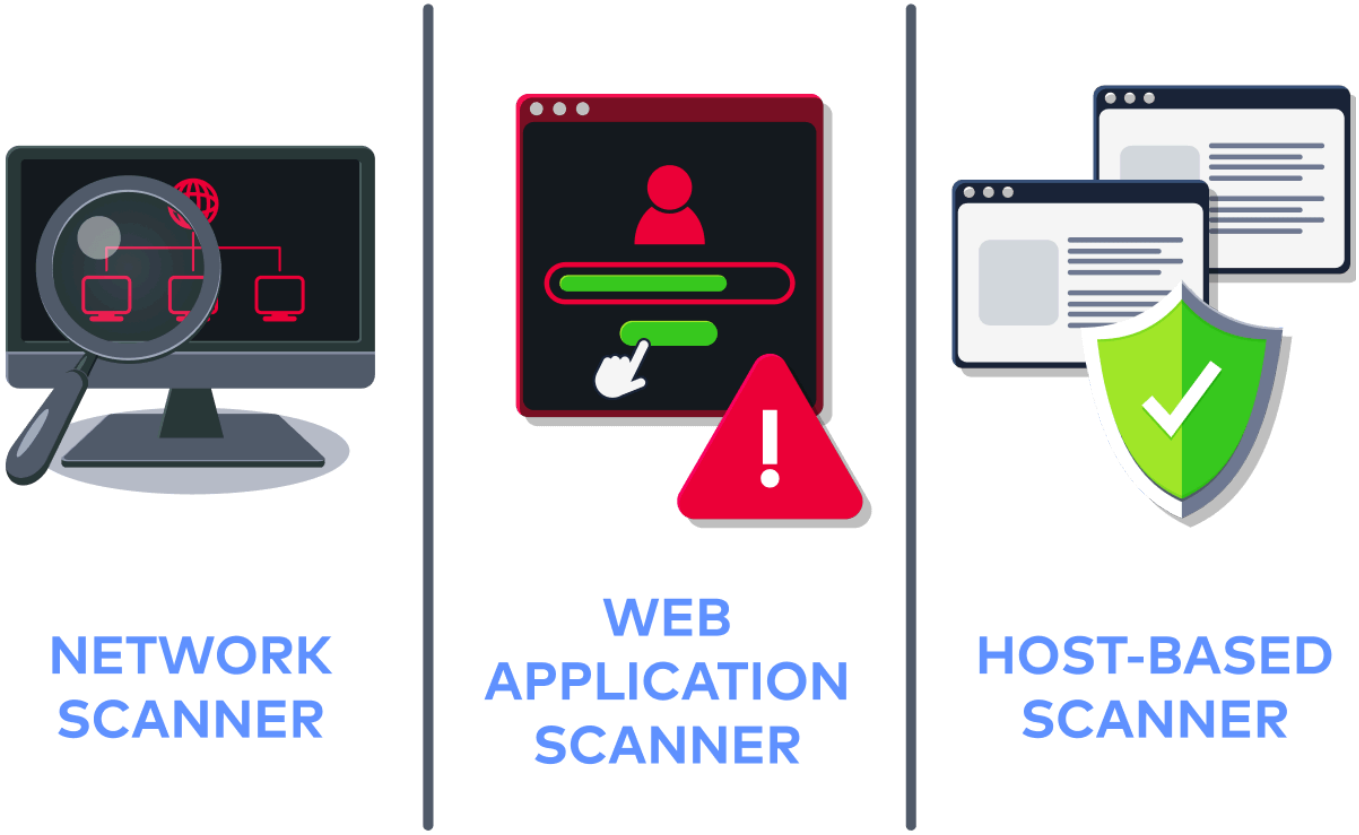
# Vulnerability Scanning Tools

# Important Topics

## Types of Vulnerability Scanners




**Network Scanner**

A network scanner examines devices on a network to identify open ports, running services, and possible exposure points. It helps map out what systems are accessible and where risks may exist. For example, walking down a street and checking which doors or windows are unlocked

**Web Application Scanner**

A web application scanner analyses websites or web apps for security issues, including outdated components, weak configurations, or risky behaviours. It focuses specifically on the HTTP/HTTPS layer. For example, testing a login form to see if it accepts weak or unexpected inputs.

**Host-Based Scanner**

A host-based scanner runs directly on a system to check for missing patches, outdated software, and insecure configuration settings. It provides a detailed view of internal system health. For example, similar to an antivirus that reviews installed programs for known issues, not just malware.

## Output of a Scanning Report

- Include the severity level, indicating how serious the issue is
- Detected services and version information
- Related CVEs or known risks
- Recommended fixes or mitigation steps

# Nmap

Network Mapper (Nmap) is a widely used network scanning tool that discovers hosts, identifies open ports, and determines which services are running on systems.

## Host Discovery

Host discovery determines which devices on a network are active and responding. This step helps build a list of live systems to focus on before deeper scanning.

nmap -sn [ip address]

```
ubuntu@tryhackme:~$ nmap -sn 10.130.128.140
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-18 17:38 UTC
Nmap scan report for ip-10-130-128-140.eu-west-3.compute.internal (10.130.128.14
0)
Host is up (0.00019s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.00 seconds
```

## Port Scanning

Port scanning identifies which ports are open and what communication pathways are available. Open ports often reveal running services that may contain vulnerabilities or misconfigurations.

```
ubuntu@tryhackme:~$ sudo nmap -sC -sV -p- 10.130.128.140
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-18 17:34 UTC
Nmap scan report for ip-10-130-128-140.eu-west-3.compute.internal (10.130.128.14
0)
Host is up (0.000013s latency).
Not shown: 65530 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.5
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:10.130.128.140
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
```

## Aggressive Scan

An aggressive scan performs deeper enumeration, including OS detection, service versions, running scripts, and traceroute. It provides far more detailed information than basic scans, but it also generates louder, more noticeable traffic.

## Vulnerability Scanning via NSE

Nmap includes the Nmap Scripting Engine (NSE), a powerful framework that automates vulnerability checks. NSE scripts can test for common issues, weak configurations, outdated services, and more, enabling Nmap to perform lightweight vulnerability assessments.

```
ubuntu@tryhackme:~$ nmap --script=ftp-anon -p21 10.130.128.140
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-18 17:42 UTC
Nmap scan report for ip-10-130-128-140.eu-west-3.compute.internal (10.130.128.14
0)
Host is up (0.00025s latency).

PORT   STATE SERVICE
21/tcp open  ftp
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)

Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds
```

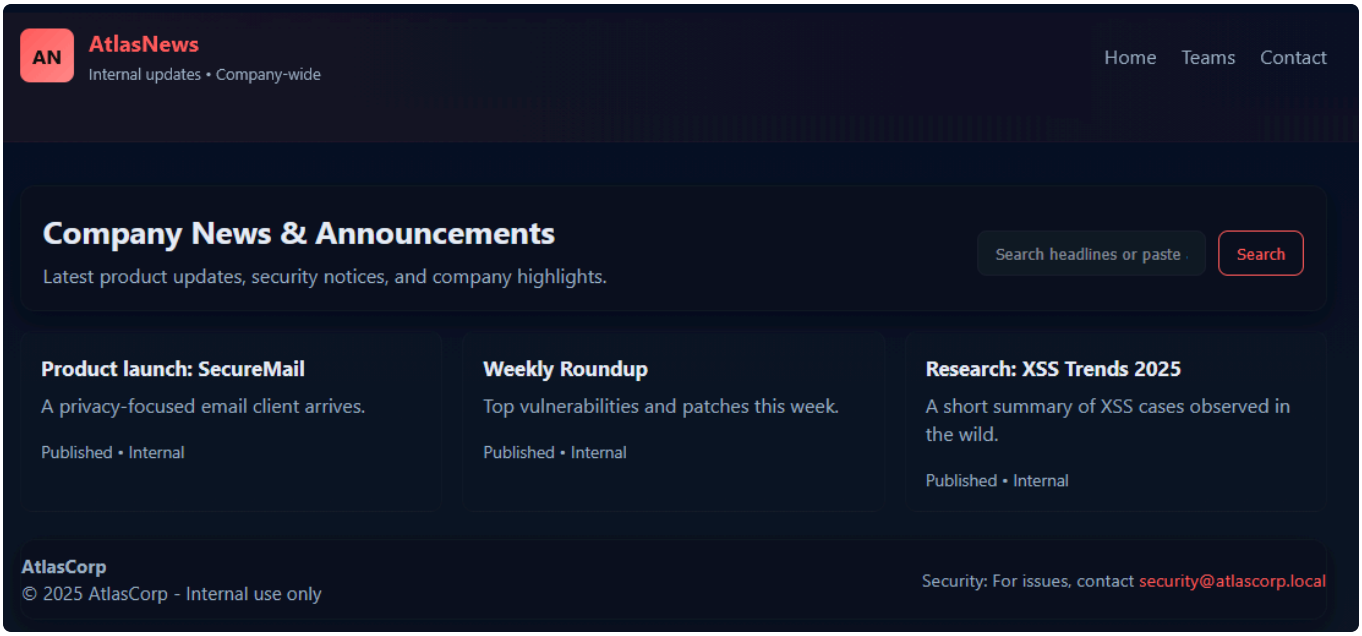
The Nmap Scripting Engine allows Nmap to run specialised scripts written in Lua to extend its functionality. These scripts can perform tasks such as brute-force attempts, version checks, protocol analysis, and basic vulnerability detection. NSE makes Nmap more than just a port scanner; it becomes a flexible tool for in-depth exploration and early security assessment.

# Nikto Web Scanner

Nikto is a widely used web server vulnerability scanner that identifies insecure configurations, outdated components, and common web-based weaknesses. It performs a comprehensive scan of a target web server and checks it against thousands of known issues, including default files, dangerous HTTP methods, misconfigurations, and potential security exposures.

## Scanning the Environment

nikto -h http://10.130.128.140:8080




```
ubuntu@tryhackme:~$ nikto -h http://10.130.128.140:8080
- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          10.130.128.140
+ Target Hostname:    ip-10-130-128-140.eu-west-3.compute.internal
+ Target Port:        8080
+ Start Time:         2026-08-18 18:16:49 (GMT0)
---------------------------------------------------------------------------
+ Server: Apache/2.4.58 (Ubuntu)
+ Cookie PHPSESSID created without the httponly flag
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ DEBUG HTTP verb may show server debugging information. See http://msdn.microsoft.com/en-us/library/e
8z01xdh%28VS.80%29.aspx for details.
+ OSVDB-561: /server-status: This reveals Apache information. Comment out appropriate line in httpd.co
nf or restrict access to allowed hosts.
+ OSVDB-3233: /info.php: PHP is installed, and a test script which runs phpinfo() was found. This give
s a lot of system information.
+ OSVDB-3268: /static/: Directory indexing found.
+ OSVDB-5292: /info.php?file=http://cirt.net/rfiinc.txt?: RFI from RSnake's list (http://ha.ckers.org/
weird/rfi-locations.dat) or from http://osvdb.org/
+ 6544 items checked: 0 error(s) and 7 item(s) reported on remote host
+ End Time:           2026-08-18 18:16:56 (GMT0) (7 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

## Understanding the Nikto Scan Results

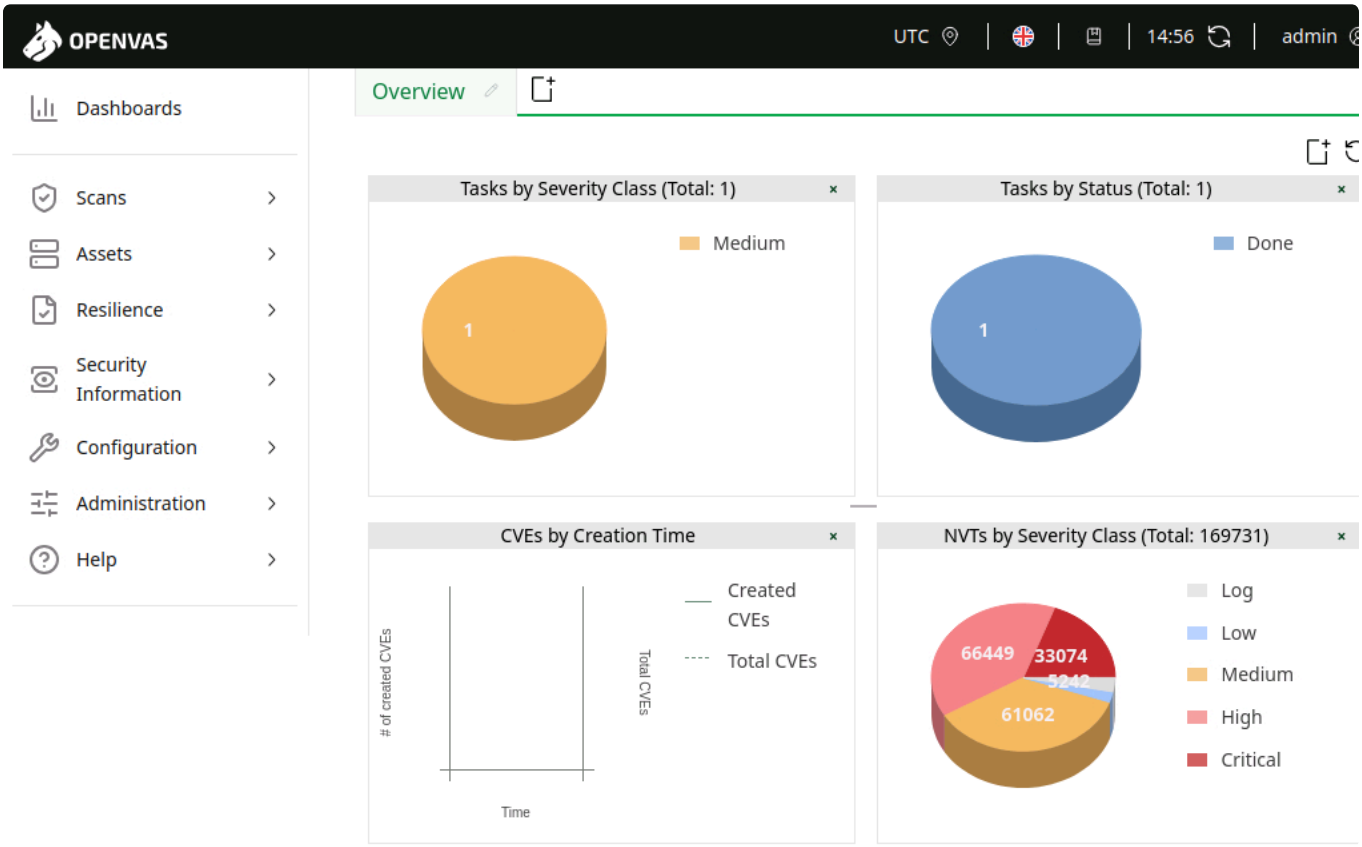
the scanner identifies several issues related to insecure configurations, unnecessary information exposure, and web server misconfigurations. These findings highlight potential

entry points attackers might explore.

- **Web Server Information Disclosure (Server: Apache/2.4.58)**: Nikto detects the exact server version, which helps attackers look up known vulnerabilities for that version. Ideally, servers should limit the amount of version information to reduce unnecessary exposure.
- **Cookie Missing Security Flags (PHPSESSID created without the HttpOnly flag)**: This means the cookie can be accessed via client-side scripts, increasing the risk of session hijacking. Secure applications normally set both the HttpOnly and Secure flags on cookies used to handle session data.
- **Missing Clickjacking Protection (X-Frame-Options header not present)**: Without this header, the site can be embedded in an iframe, enabling “clickjacking” attacks that trick users into clicking hidden elements.
- **Potentially Dangerous HTTP Method (DEBUG HTTP verb available)**: Some servers respond to uncommon methods, such as DEBUG, which can reveal internal debugging information. This is unnecessary for normal operation and should be disabled to reduce information leakage.
- **Exposed PHP Info Page (/info.php found running phpinfo())**: The `phpinfo()` output reveals extensive system information, modules, configuration values, and environment variables and is commonly removed in production environments. Leaving it accessible introduces high information leakage risks.
- **Directory Indexing Enabled (/static/ directory indexing)**: This allows anyone to browse the contents of a directory without restriction, revealing files that shouldn’t be publicly visible. Directory listing should be disabled unless absolutely required.
- **Remote File Inclusion (RFI) Test Detected (info.php?file=<external_link>)**: Nikto detected a parameter that may be vulnerable to Remote File Inclusion if mishandled. RFI vulnerabilities allow attackers to load code from external servers and execute it on the target system, one of the most severe web risks.

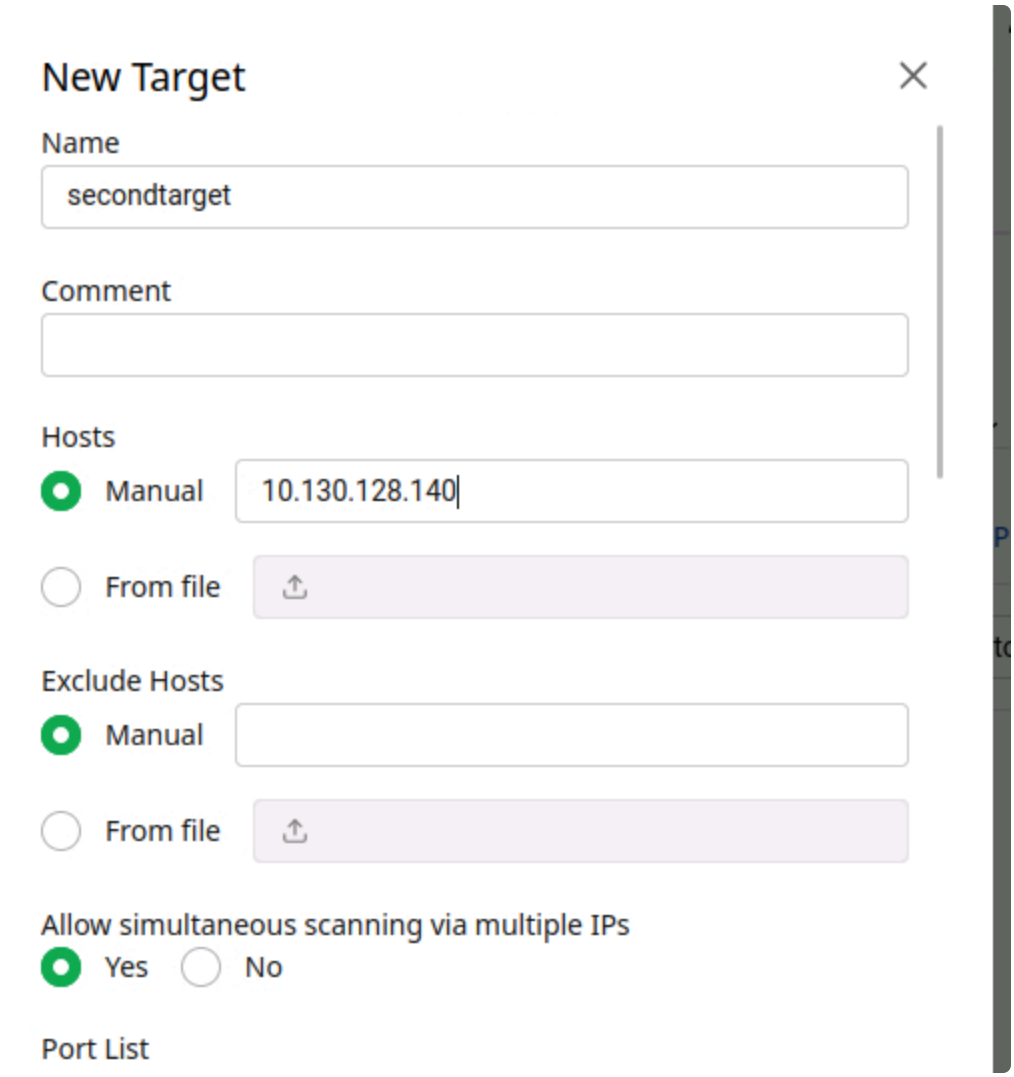
# OpenVas & Greenbone Vulnerability Management

OpenVAS, a powerful vulnerability scanner that assesses both host-level weaknesses and web application security issues. It provides deep, authenticated and unauthenticated scanning, making it ideal for identifying misconfigurations, outdated software, missing patches, and exposed services.




## Adding a Target

OpenVAS requires a target to be defined. Navigate to `Configuration > Targets`, click the `Plus (+)` icon, and enter the target name `SecondTarget` and the IP `10.130.128.140`. Once the details are filled in, click `Save` to register the target for scanning, as shown below:

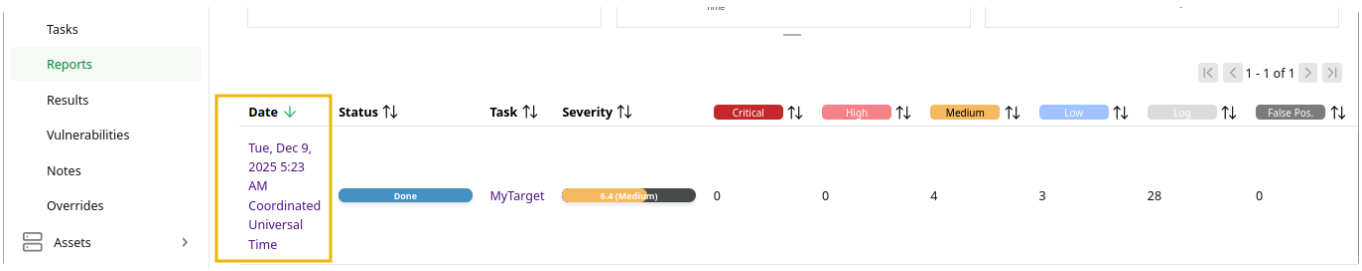



## Scanning a Target

Go to `Scans > Tasks`, click the `Plus (+)` icon, and choose `New Task`. Enter the task name `SecondTask` and select the target you added earlier, keep the default scan configuration, and click `Save` to create the task. Once created, click the `Run` button to begin the vulnerability scan. OpenVAS will analyse the host and display real-time progress until the scan completes.

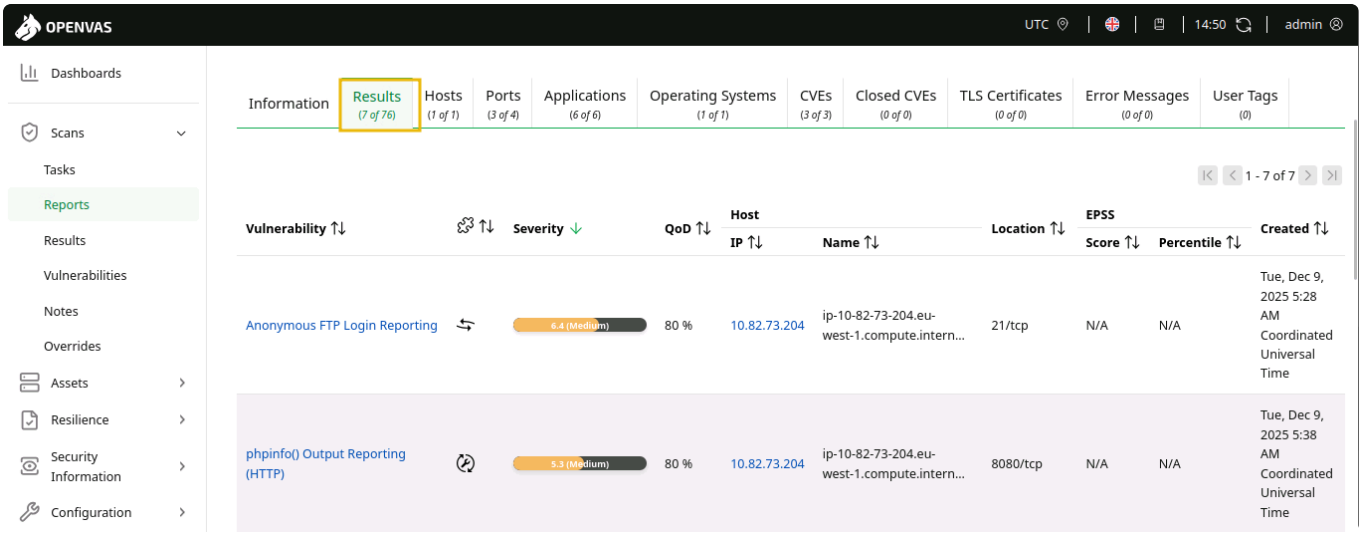
## Viewing the Report

To view the results of the `MyTarget` report, go to `Scans > Reports`. Click the report entry for the scan date and time.



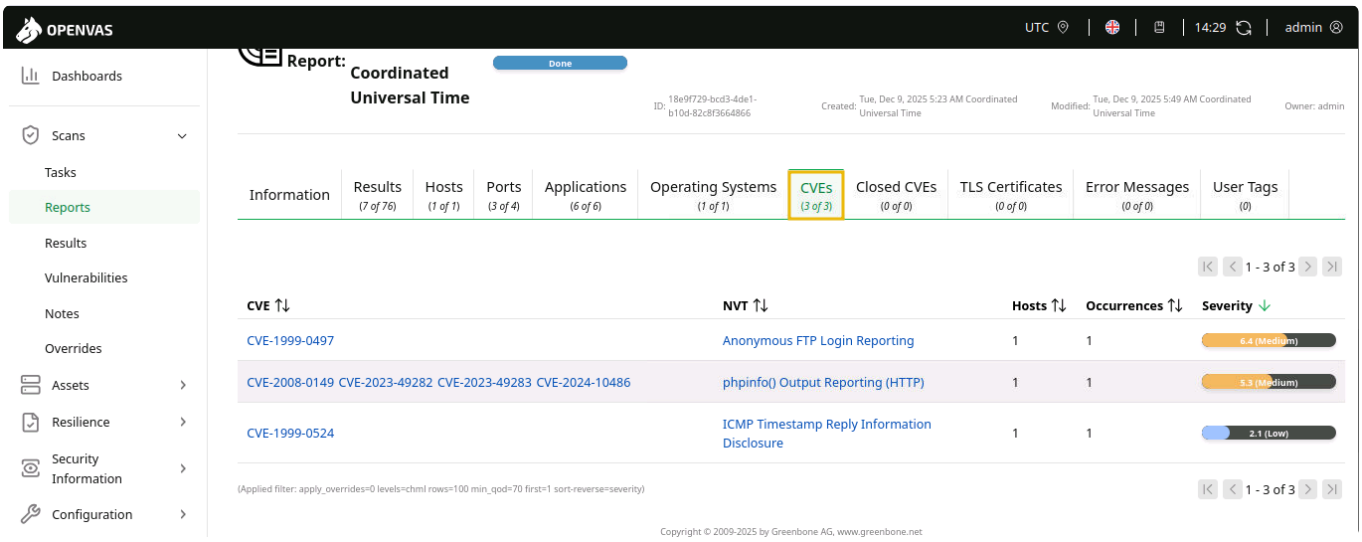

**Results Tab**

In the Results tab, the scan findings are listed by severity level, affected ports, and host details. The report highlights issues such as anonymous FTP access, exposed phpinfo pages, missing cookie protections, and weak encryption settings




### CVEs Tab

In the CVEs tab, OpenVAS maps detected issues to known published vulnerabilities. The scan identified several CVEs, including ones related to anonymous FTP access, phpinfo exposure, and ICMP timestamp disclosure.




# Best Practices

- **Start Light:** Begin with non-intrusive scans to avoid slowing or disrupting the target system, especially in shared or production environments.
- **Right Tooling:** Match the scanner to the job—use Nmap for network discovery, OpenVAS for deep analysis, and Nikto for web server checks.
- **Verify Results:** Double-check high-severity findings manually because scanners can produce false positives or incomplete information.

- **Stay Updated**: Keep scanning tools and vulnerability feeds up to date so the scanner can identify new CVEs and configuration issues.
- **Use Credentials:** When allowed, run authenticated scans to get more accurate visibility into missing patches and internal weaknesses.
- **Compare Over Time:** Save scan outputs and compare them regularly to track improvements, detect regressions, and monitor newly introduced issues.
- **Scan Smart:** Schedule scans during low-traffic hours since vulnerability scanning can consume noticeable network and system resources.